

Name: Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC Day-23 — Increasing Patterns in C

1. Algorithm

Step 1: Start the program.

Step 2: Display the menu containing five increasing-pattern options and an Exit option.

Step 3: Ask the user to enter a choice.

Step 4: Use a `switch` statement to process the choice.

Step 5: Call the corresponding parameterless function.

Step 6: Inside the selected function, input the number of rows `n`.

Step 7: Use nested `for` loops to generate the required pattern.

Step 8: For the alphabet pattern, ensure that `n` is between 1 and 26.

Step 9: If the menu choice is invalid, display an error message.

Step 10: Repeat the menu until the user selects Exit.

Step 11: Stop.

The five required patterns are the right-angled asterisk triangle, row-number triangle, natural-number pattern, consecutive-number triangle, and alphabet pattern.

2. C Program

```
#include <stdio.h>

void starTriangle();
void rowNumberTriangle();
void naturalNumberTriangle();
void consecutiveTriangle();
void alphabetTriangle();

int main()
{
    int choice;
```

```

do
{
    printf("\n--- INCREASING PATTERNS MENU ---\n");
    printf("1. Increasing right-angled triangle\n");
    printf("2. Increasing row-number triangle\n");
    printf("3. Increasing natural-number pattern\n");
    printf("4. Consecutive-number triangle\n");
    printf("5. Increasing alphabet pattern\n");
    printf("6. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            starTriangle();
            break;

        case 2:
            rowNumberTriangle();
            break;

        case 3:
            naturalNumberTriangle();
            break;

        case 4:
            consecutiveTriangle();
            break;

        case 5:
            alphabetTriangle();
            break;

        case 6:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice! Please enter 1 to 6.\n");
    }

    } while(choice != 6);

    return 0;
}

/* Pattern 1 */
void starTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

```

```

        for(i = 1; i <= n; i++)
        {
            for(j = 1; j <= i; j++)
            {
                printf("* ");
            }
            printf("\n");
        }
    }

/* Pattern 2 */
void rowNumberTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

/* Pattern 3 */
void naturalNumberTriangle()
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

/* Pattern 4 */
void consecutiveTriangle()
{
    int n, i, j;
    int num = 1;

    printf("Enter number of rows: ");

```

```

scanf("%d", &n);

for(i = 1; i <= n; i++)
{
    for(j = 1; j <= i; j++)
    {
        printf("%d ", num);
        num++;
    }
    printf("\n");
}

/* Pattern 5 */
void alphabetTriangle()
{
    int n, i, j;

    printf("Enter number of rows (1-26): ");
    scanf("%d", &n);

    if(n < 1 || n > 26)
    {
        printf("Invalid input! Enter rows between 1 and 26.\n");
        return;
    }

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= i; j++)
        {
            printf("%c ", 'A' + j - 1);
        }
        printf("\n");
    }
}

```

3. Explanation of Each Pattern

Pattern 1 — Increasing Right-Angled Triangle

The PDF asks for an increasing right-angled triangle made using asterisks.

For $n = 4$:

```

*
* *
* * *
* * * *

```

Logic

```
for(i = 1; i <= n; i++)
{
    for(j = 1; j <= i; j++)
        printf("* ");
}
```

The **outer loop controls the rows**, while the **inner loop prints stars**. Row *i* contains exactly *i* stars.

Pattern 2 — Increasing Row-Number Triangle

According to the question, each row number must be printed as many times as its value.

For *n* = 4:

```
1
2 2
3 3 3
4 4 4 4
```

Logic

```
printf("%d ", i);
```

Here *i* represents the row number.

Therefore:

- Row 1 → 1 printed once
 - Row 2 → 2 printed twice
 - Row 3 → 3 printed three times
 - Row 4 → 4 printed four times
-

Pattern 3 — Increasing Natural-Number Pattern

The question requires natural numbers starting from 1 up to the corresponding row number.

For *n* = 4:

```
1
1 2
```

```
1 2 3
1 2 3 4
```

The inner-loop variable `j` is printed:

```
printf("%d ", j);
```

So every new row starts again from 1.

Pattern 4 — Consecutive-Number Triangle

Here the numbers must remain consecutive throughout the entire triangle.

For `n = 4`:

```
1
2 3
4 5 6
7 8 9 10
```

We initially declare:

```
int num = 1;
```

After printing a number:

```
num++;
```

Therefore, the number continues increasing rather than restarting on every row.

Pattern 5 — Increasing Alphabet Pattern

The PDF specifies uppercase English letters and limits `n` to **1 through 26**.

For `n = 4`:

```
A
A B
A B C
A B C D
```

The important statement is:

```
printf("%c ", 'A' + j - 1);
```

For example:

```
j = 1 → A + 0 → A
j = 2 → A + 1 → B
j = 3 → A + 2 → C
j = 4 → A + 3 → D
```

4. Test-Case Table

Test Case	Menu Choice	Input n	Expected Result	Actual Result	Status
1	1	4	Increasing star triangle with 4 rows	Same as expected	Pass
2	2	4	Row numbers repeated 1–4 times	Same as expected	Pass
3	3	4	Natural numbers from 1 to each row number	Same as expected	Pass
4	4	4	Consecutive numbers 1–10 in triangle	Same as expected	Pass
5	5	4	Alphabet triangle from A to D	Same as expected	Pass
6	5	27	Invalid input message	Invalid input displayed	Pass
7	8	—	Invalid menu choice message	Invalid choice displayed	Pass
8	6	—	Program terminates	Program terminated	Pass

5. Evaluation Against Individual Test Cases

Test Case 1 – Increasing Right-Angled Triangle:

The program was tested with $n = 4$. It correctly printed 1, 2, 3, and 4 stars in successive rows.

```
(kali㉿kali)-[~/Desktop]
$ cc day23.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

— INCREASING PATTERNS MENU —
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 1
Enter number of rows: 4
*
* *
* * *
* * * *
```

Result: PASS.

Test Case 2 – Increasing Row-Number Triangle:

The program was tested with $n = 4$. Each row number was correctly printed as many times as its value.

```
Enter your choice: 2
Enter number of rows: 4
1
2 2
3 3 3
4 4 4 4
```

Result: PASS.

Test Case 3 – Increasing Natural-Number Pattern:

For $n = 4$, the program correctly printed natural numbers from 1 up to the corresponding row number.

```
Enter your choice: 3
Enter number of rows: 4
1
1 2
1 2 3
1 2 3 4
```

Result: PASS.

Test Case 4 – Consecutive-Number Triangle:

For $n = 4$, the program correctly printed consecutive numbers from 1 to 10 in triangular form.

```
Enter your choice: 4
Enter number of rows: 4
1
2 3
4 5 6
7 8 9 10
```

Result: PASS.

Test Case 5 – Increasing Alphabet Pattern:

For $n = 4$, the program correctly generated the alphabet pattern from A to D across four rows.

```
Enter your choice: 5
Enter number of rows (1-26): 4
A
A B
A B C
A B C D
```

Result: PASS.

Test Case 6 – Invalid Alphabet Input:

The program was tested with $n = 27$. Since the allowed range is 1-26, it correctly displayed an invalid-input message.

```
Enter your choice: 5
Enter number of rows (1-26): 27
Invalid input! Enter rows between 1 and 26.
```

Result: PASS.

Test Case 7 – Invalid Menu Choice:

An invalid menu choice was entered. The program correctly displayed an error message instead of executing a pattern.

```
Enter your choice: 8
Invalid choice! Please enter 1 to 6.
```

Result: PASS.

Test Case 8 – Exit:

The Exit option was selected. The program terminated correctly without executing another pattern.

```
Enter your choice: 6
Exiting program...
```

Result: PASS.